Modelado de Datos con Estructuras (`struct`) y Punteros

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 5 — Modelado de datos con estructuras y punteros **Versión del documento:** 1.0.0 **Fecha:** 2026

Introducción

En las actividades anteriores las variables se declaraban de forma suelta: un `int edad`, un `char nombre[50]`, un `float metrica`. Funciona, pero no representa el mundo real. En la realidad, esos tres datos no existen por separado: **pertenecen a la misma entidad**.

Esta actividad consiste en modelar entidades complejas del mundo real —sensores, módulos de hardware, usuarios, entidades de un proyecto— utilizando **estructuras de datos (`struct`)** en C++, controlando el ingreso y la modificación de sus miembros por referencia mediante punteros y el operador flecha (`->`).

La conexión con el Proyecto Integrador Anual es directa: la estructura desarrollada en esta actividad es, en esencia, el esqueleto de las variables principales del prototipo que el grupo presenta en la Expotécnica.

Desarrollo

El problema del agrupamiento de datos

Un `struct` es una **colección de variables de distintos tipos agrupadas bajo una misma entidad lógica**, en bloques contiguos de memoria RAM. El nombre `struct` viene de *structure*: una estructura.

```
struct EntidadProyecto
{
    int   id;
    char  nombre[50];
    float metrica;   // Lecturas de sensores, consumo, avance, etc.
};
```

1. Heterogeneidad: la ventaja de agrupar

La respuesta a por qué conviene un `struct` antes que variables sueltas es que **las variables heterogéneas que describen una misma entidad no pueden viajar juntas**. Si el `id`, el `nombre` y la `metrica` se declaran sueltas, cada función que necesite los tres tiene que recibir tres parámetros:

```
void mostrar(int id, char nombre[], float metrica);   // Tedioso y frágil
```

Con la estructura, se declara **una sola entidad** y todos sus miembros viajan juntos en un solo bloque contiguo de RAM. La función pasa a necesitar un único parámetro, y el compilador garantiza que id, nombre y metrica **siempre llegan de la misma entidad y nunca desalineados**. Ese es el beneficio real: no es solo menos código, es **integridad de datos**.

Además, el bloque contiguo mejora el acceso a memoria: como los tres miembros son cercanos entre sí, el procesador los trae en la misma línea de caché, en lugar de acceder a tres zonas dispersas de la RAM.

2. El operador flecha (`->`)

Cuando una variable es de tipo `struct`, los campos se acceden con el **punto**:

```
miEntidad.id;      // Correcto: miEntidad NO es puntero
```

Cuando la variable es de tipo **puntero a struct**, el punto deja de funcionar y hay que usar el **operador flecha**:

```
ptr->id;           // Correcto: ptr es un puntero
ptr.id;            // ERROR DE COMPILACIÓN
```

El compilador de C++ no permite usar la notación de punto cuando la variable es de tipo puntero por una razón de seguridad y de coherencia del lenguaje. `p` es un puntero, es decir, **contiene una dirección, no una estructura**. Si se escribiera `p.id`, el compilador interpretaría que se busca un campo llamado `id` *dentro de la dirección misma*, lo cual es semánticamente imposible: una dirección de memoria no tiene campos. El operador `->` es simplemente una notación más clara para el compilador que significa "desreferenciame y buscame el campo", es decir, `(*p).id`.

El operador flecha existe precisamente porque C++ es un lenguaje pensado para el uso de estructuras: si hubiera que escribir `(*ptr).nombre` en todas partes, el código sería ilegible.

3. El pasaje por referencia mediante punteros

La función `cargarDatos` no recibe la estructura **por valor**, sino **por dirección**:

```
void cargarDatos(EntidadProyecto* ptr);   // Puntero
cargarDatos(&miEntidad);                  // Se pasa la dirección
```

La razón es la **eficiencia en RAM**. Un pasaje por copia obligaría al sistema a duplicar la estructura entera en memoria, reservando un segundo bloque contiguo para que la función trabaje sobre una copia que se descarta al terminar. Con un `float`, `int` y 50 caracteres, el desperdicio es despreciable; con una estructura que contenga, por ejemplo, la imagen completa de un sensor o un historial de muestras, la copia sería enorme.

El pasaje por referencia resuelve el problema de raíz: se envía **la dirección (`&variable`) de un objeto a una función para modificar el original sin crear duplicados pesados en la RAM**. Como `cargarDatos` trabaja sobre el bloque original, todas las modificaciones quedan directamente en `miEntidad`.

El prototipo de la función lo deja explícito: `EntidadProyecto* ptr` dice "espero una dirección, no un objeto".

4. El búfer de entrada y `cin.ignore()`

Este es el detalle más sutil de la actividad y el que más errores causa en la práctica.

```
cin >> ptr->id;
cin.ignore();                          // Limpieza del búfer
cin.getline(ptr->nombre, 50);          // Ahora sí funciona
```

El **búfer de entrada** es el espacio temporal en memoria donde el sistema guarda los caracteres ingresados por teclado. Cuando el usuario escribe `42` y presiona Enter, quedan dos cosas en el

búfer: los caracteres `4` y `2` —consumidos por `cin >>`— y el carácter de salto de línea `\n` —**pendiente**.

Ese `\n` remanente es el problema. `cin.getline()` lee hasta el próximo `\n`, así que al invocarlo de inmediato encuentra el `Enter` sobrante de la lectura anterior y devuelve **una cadena vacía**, sin darle oportunidad al usuario de escribir nada.

`cin.ignore()` resuelve el problema descartando ese carácter pendiente del búfer. Sin esa línea, el campo `nombre` queda en blanco aunque el usuario haya escrito correctamente su nombre, y el error es desconcertante porque el programa no falla: simplemente muestra una cadena vacía.

Además, `cin.getline(ptr->nombre, 50)` recibe el tamaño del arreglo como segundo parámetro justamente para no escribir más allá de los 50 caracteres reservados en la RAM, previniendo un desbordamiento del búfer.

5. Verificación final desde `main()`

Una vez que `cargarDatos` terminó, `main()` lee los miembros con notación de punto —porque `miEntidad` **no** es un puntero— y muestra la dirección de memoria con el operador `&`:

```
cout << "ID Registrado: "      << miEntidad.id      << endl;
cout << "Nombre Registrado: "  << miEntidad.nombre  << endl;
cout << "Metrica Guardada: "   << miEntidad.metrica << endl;
cout << "Direccion RAM Hexadecimal: " << &miEntidad  << endl;
```

Mostrar la dirección de la estructura completa demuestra que `id`, `nombre` y `metrica` viven efectivamente en un **único bloque contiguo**, que es justamente lo que un `struct` garantiza.

6. Aplicación al Proyecto Integrador Anual

La estructura `EntidadProyecto` puede modelar cualquier entidad del prototipo grupal. Por ejemplo, si el proyecto es un sistema de monitoreo, `id` sería el número de serie del sensor, `nombre` su ubicación o descripción y `metrica` la última lectura registrada. Si en cambio el proyecto gestiona turnos o stock, la misma estructura sirve con otros nombres de campo y otros tipos.

Conclusión

La actividad consolidó el paso de "variables sueltas" a "entidades modeladas", que es el mismo salto que hace años daría el paso a la programación orientada a objetos.

Las tres ideas principales son: el `struct` existe para **agrupar lo que en el mundo real está junto**, y esa agrupación no es solo estética sino que garantiza integridad y caché; el **operador flecha** no es decorativo, es la consecuencia lógica de que un puntero contenga una dirección y no una estructura; y el **pasaje por referencia** es la herramienta que hace que manipular entidades sea eficiente en RAM en lugar de caro.

El detalle de `cin.ignore()` parece menor, pero resume una verdad de la programación de bajo nivel: los errores más difíciles de encontrar no son lógicos, son de **estado oculto**. El programa funciona, no falla, y sin embargo el resultado es incorrecto. Solo se detecta entendiendo qué hay realmente en memoria en cada momento.

Referencias

• cppreference.com. (2024). *Struct*. C++ reference.

<https://en.cppreference.com/w/cpp/language/struct>

• cppreference.com. (2024). *Member access operators* `->` *and* `.`. C++ reference.
<https://en.cppreference.com/w/cpp/language/member_access>

• Microsoft. (2024). *Pointers (C++)*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/pointers-cpp>

• Microsoft. (2024). *cin, cout, cerr, clog and related streams*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/iostream>

• GeeksforGeeks. (2024). *Structures in C++*. <https://www.geeksforgeeks.org/structures-in-cpp/>